

PRD Técnico Consolidado - Tá na Rua!

Back-end, banco de dados, regras de negócio e validação

Responsável: Integrante 2

Versão consolidada 1.2 - 18 de julho de 2026

Controle do documento

Projeto: Tá na Rua!

Módulo: Back-end, banco de dados e regras de negócio

Responsável: Integrante 2

Tipo: MVP acadêmico funcional

Arquitetura: API REST modular

Versão consolidada: 1.2

Data: 18 de julho de 2026

Histórico de versões

1.0: escopo técnico, arquitetura, entidades, regras, endpoints, segurança, testes e fases.

1.1: decisões técnicas definitivas, correções de modelagem, máquina de estados, prioridade, privacidade e escopo do MVP.

1.2: interfaces de desenvolvimento, Swagger, Adminer, seeds, coleção de API, scripts e validação por fase.

Ordem de prevalência em caso de conflito: versão 1.2, versão 1.1 e, por último, versão 1.0.

Sumário executivo

Parte I - PRD Técnico 1.0: contexto, escopo, arquitetura, entidades, regras, API, segurança, testes, fases e entregáveis.

Parte II - Adendo 1.1: decisões definitivas de stack, modelagem corrigida, autenticação, protocolo, máquina de estados, prioridade, privacidade e escopo do MVP.

Parte III - Adendo 1.2: Swagger, Adminer, seeds, coleção de API, scripts e validação manual e automatizada.

Parte IV - Checklist mestre: controle de progresso das fases 0 a 10, evidências e regra de aprovação.

Apêndices: portas, acessos e comandos principais.

Parte I - PRD Técnico 1.0

1. Contexto do produto

O **Tá na Rua!** é uma plataforma de inteligência urbana colaborativa que permite aos cidadãos registrar falhas de infraestrutura, como:

buracos;

postes apagados;

vazamentos;

lixo acumulado;

calçadas danificadas;

esgoto;

alagamentos;

sinalização danificada.

O cidadão envia uma imagem, localização e descrição. A inteligência artificial sugere a categoria e a gravidade do problema. O sistema procura registros semelhantes, agrupa ocorrências duplicadas e permite que outros moradores confirmem o problema usando a função **“Eu também vi”**.

A prefeitura utiliza um painel para consultar os problemas priorizados, alterar o status do atendimento e registrar a conclusão do reparo. Após a conclusão, os cidadãos podem avaliar a qualidade do serviço.

O back-end será o núcleo que conecta:

- aplicativo do cidadão;
- módulo de inteligência artificial;
- painel da prefeitura;
- sistema de notificações;
- Telegram, WhatsApp e n8n;
- banco de dados.

2. Objetivo do back-end

Desenvolver uma API segura e documentada que permita:

- cadastrar e autenticar usuários;
- armazenar ocorrências urbanas;
- armazenar imagens e localização;
- consultar problemas em um mapa;
- registrar confirmações comunitárias;
- identificar possíveis ocorrências próximas;
- integrar o sistema ao módulo de IA;
- controlar alterações de status;
- manter o histórico das ocorrências;
- registrar avaliações dos reparos;
- fornecer dados para o painel municipal;
- gerar notificações internas;
- registrar ações administrativas;
- proteger dados pessoais e controlar permissões.

3. Responsabilidades do Integrante 2

3.1 Incluído no escopo

- criação do projeto back-end;
- configuração do ambiente;
- modelagem do banco de dados;
- migrations;
- seeds de desenvolvimento;
- API REST;
- validação dos dados;

cadastro de usuários;
login e autenticação;
controle de sessão;
autorização por perfil;
CRUD de ocorrências;
upload e armazenamento de imagens;
consultas por localização;
sistema “Eu também vi”;
histórico de status;
avaliações de reparo;
notificações internas;
logs e auditoria;
endpoints do painel;
integração com o módulo de IA;
integração com n8n;
documentação OpenAPI/Swagger;
testes do back-end;
tratamento centralizado de erros.

3.2 Fora do escopo

criação das telas do cidadão;
criação dos gráficos do painel;
treinamento do modelo de IA;
design da interface;
landing page;
vídeo demonstrativo;
automações completas do WhatsApp;
implantação de GOV.BR;
ranking nacional real;
cálculo oficial de custos de obras;
desenvolvimento de aplicativo Android ou iOS nativo.

4. Tecnologias recomendadas

4.1 Linguagem e servidor

Node.js;
TypeScript;
Express;
arquitetura modular por domínio.

4.2 Banco de dados

PostgreSQL;
extensão PostGIS para consultas geográficas;
migrations versionadas.

4.3 Camada de acesso ao banco

Sugestão principal:
Drizzle ORM para tabelas e consultas comuns;
SQL parametrizado para consultas PostGIS mais específicas.
Alternativa mais simples:
Knex.js para migrations e consultas;
driver pg para PostgreSQL.

4.4 Autenticação e segurança

JWT de curta duração;
refresh token;
hash de senha com Argon2 ou bcrypt;
cookies httpOnly para refresh token;
controle de acesso baseado em perfil;
rate limiting;
Helmet;
CORS configurado;
validação com Zod.

4.5 Imagens

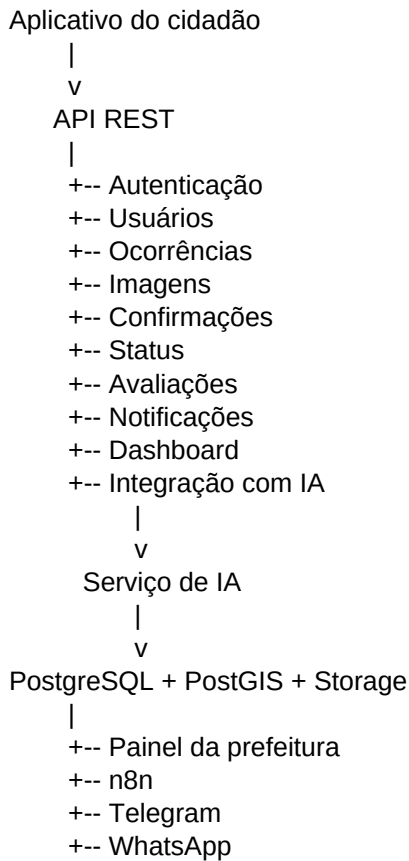
Para o MVP:
Supabase Storage;
Cloudflare R2;
Amazon S3;
outro serviço compatível com S3.
O banco não deverá guardar a imagem diretamente. Ele deverá armazenar:
URL;
chave do arquivo;
tipo;
tamanho;
proprietário;
data de envio.

4.6 Documentação e testes

Swagger/OpenAPI;

Vitest;
 Supertest;
 ESLint;
 Prettier;
 Docker Compose.

5. Arquitetura proposta



6. Perfis de acesso

6.1 CITIZEN

Cidadão comum.

Pode:

cadastrar ocorrência;
 consultar ocorrências públicas;
 confirmar uma ocorrência;
 adicionar imagem;
 acompanhar seus registros;
 avaliar um reparo;
 consultar notificações.

6.2 CITY_OPERATOR

Operador autorizado da prefeitura.

Pode:

- consultar ocorrências da cidade;
- alterar status;
- definir setor responsável;
- informar prazo;
- registrar conclusão;
- anexar comprovantes;
- consultar indicadores municipais.

6.3 MODERATOR

Moderador da plataforma.

Pode:

- revisar denúncias;
- corrigir categorias;
- analisar conteúdo indevido;
- unir ocorrências;
- separar ocorrências agrupadas;
- rejeitar registros;
- revisar decisões da IA.

6.4 ADMIN

Administrador geral.

Pode:

- executar todas as ações;
- gerenciar usuários;
- gerenciar municípios;
- gerenciar categorias;
- consultar logs;
- configurar regras de prioridade.

7. Entidades do banco de dados

7.1 users

Representa cidadãos, operadores, moderadores e administradores.

id	UUID
name	VARCHAR
email	VARCHAR UNIQUE
phone	VARCHAR UNIQUE NULL
password_hash	VARCHAR

role ENUM
 municipality_id UUID NULL
 neighborhood VARCHAR NULL
 avatar_url VARCHAR NULL
 status ENUM
 email_verified_at TIMESTAMP NULL
 last_login_at TIMESTAMP NULL
 created_at TIMESTAMP
 updated_at TIMESTAMP
 deleted_at TIMESTAMP NULL

Status:

ACTIVE;

PENDING;

BLOCKED;

DELETED.

7.2 municipalities

id UUID
 name VARCHAR
 state CHAR(2)
 ibge_code VARCHAR UNIQUE
 latitude DECIMAL
 longitude DECIMAL
 active BOOLEAN
 created_at TIMESTAMP
 updated_at TIMESTAMP

7.3 categories

id UUID
 name VARCHAR
 slug VARCHAR UNIQUE
 description TEXT
 icon VARCHAR NULL
 active BOOLEAN
 created_at TIMESTAMP
 updated_at TIMESTAMP

Categorias iniciais:

buraco na via;

iluminação pública;

vazamento de água;

esgoto;

lixo;

calçada danificada;

alagamento;

árvore caída;

sinalização;

semáforo;

outro.

7.4 occurrences

Registro principal do problema urbano.

id	UUID
protocol	VARCHAR UNIQUE
title	VARCHAR
description	TEXT
category_id	UUID
municipality_id	UUID
created_by	UUID
assigned_department_id	UUID NULL
status	ENUM
severity	INTEGER
priority_score	DECIMAL
risk_level	ENUM
address	VARCHAR
neighborhood	VARCHAR
latitude	DECIMAL
longitude	DECIMAL
location	GEOGRAPHY(Point, 4326)
location_accuracy	DECIMAL NULL
anonymous_publication	BOOLEAN
citizen_count	INTEGER
confirmation_count	INTEGER
first_reported_at	TIMESTAMP
resolved_at	TIMESTAMP NULL
created_at	TIMESTAMP
updated_at	TIMESTAMP
deleted_at	TIMESTAMP NULL

Status:

PENDING_REVIEW;

PUBLISHED;

FORWARDED;

ACKNOWLEDGED;

UNDER_ANALYSIS;

SCHEDULED;

IN_PROGRESS;

RESOLVED;

CONTESTED;

CLOSED;

REJECTED;

DUPLICATE.

Risco:

LOW;

MEDIUM;

HIGH;

CRITICAL.

7.5 occurrence_reports

Armazena cada denúncia individual ligada a uma ocorrência principal. Isso permite que várias pessoas reportem o mesmo problema sem criar dezenas de ocorrências públicas.

```

id          UUID
occurrence_id  UUID
reported_by   UUID
original_description TEXT
latitude      DECIMAL
longitude     DECIMAL
location      GEOGRAPHY(Point, 4326)
reported_at   TIMESTAMP
source        ENUM
created_at    TIMESTAMP

```

Fontes:

WEB_APP;

TELEGRAM;

WHATSAPP;

ADMIN;

API.

7.6 occurrence_images

```

id          UUID
occurrence_id  UUID
report_id     UUID NULL
uploaded_by   UUID
file_url      VARCHAR
storage_key    VARCHAR
mime_type     VARCHAR
file_size     INTEGER
image_type    ENUM
moderation_status ENUM
created_at    TIMESTAMP

```

Tipos:

INITIAL;

UPDATE;

BEFORE_REPAIR;

DURING_REPAIR;

AFTER_REPAIR;

EVALUATION.

7.7 occurrence_confirmations

Representa o botão “Eu também vi”.

```

id          UUID
occurrence_id  UUID
user_id      UUID

```

directly_affected BOOLEAN
 problem_worsened BOOLEAN
 comment TEXT NULL
 created_at TIMESTAMP
 updated_at TIMESTAMP

Restrição única:

UNIQUE (occurrence_id, user_id)

O mesmo usuário não poderá confirmar duas vezes a mesma ocorrência.

7.8 occurrence_status_history

id UUID
 occurrence_id UUID
 previous_status ENUM NULL
 new_status ENUM
 changed_by UUID
 reason TEXT NULL
 public_message TEXT NULL
 created_at TIMESTAMP

Nenhuma alteração de status deverá acontecer sem criar uma entrada nessa tabela.

7.9 repair_evaluations

id UUID
 occurrence_id UUID
 user_id UUID
 rating INTEGER
 problem_resolved BOOLEAN
 service_quality INTEGER NULL
 comment TEXT NULL
 created_at TIMESTAMP
 updated_at TIMESTAMP

Regras:

nota entre 1 e 5;

apenas usuários relacionados à ocorrência podem avaliar;

cada usuário avalia uma vez;

avaliação poderá ser atualizada durante prazo configurado;

várias avaliações negativas poderão reabrir a ocorrência.

7.10 ai_analyses

id UUID
 occurrence_id UUID NULL
 report_id UUID NULL
 analysis_type ENUM
 model_name VARCHAR
 suggested_category VARCHAR NULL
 suggested_severity INTEGER NULL
 suggested_risk VARCHAR NULL
 confidence DECIMAL
 summary TEXT NULL
 possible_duplicates JSONB NULL

raw_result	JSONB
requires_human_review	BOOLEAN
reviewed_by	UUID NULL
reviewed_at	TIMESTAMP NULL
created_at	TIMESTAMP

7.11 notifications

id	UUID
user_id	UUID
type	ENUM
title	VARCHAR
message	TEXT
entity_type	VARCHAR NULL
entity_id	UUID NULL
read_at	TIMESTAMP NULL
created_at	TIMESTAMP

7.12 departments

Setores da prefeitura.

id	UUID
municipality_id	UUID
name	VARCHAR
description	TEXT NULL
active	BOOLEAN
created_at	TIMESTAMP
updated_at	TIMESTAMP

7.13 audit_logs

id	UUID
user_id	UUID NULL
action	VARCHAR
entity_type	VARCHAR
entity_id	UUID NULL
previous_data	JSONB NULL
new_data	JSONB NULL
ip_address	VARCHAR NULL
user_agent	TEXT NULL
created_at	TIMESTAMP

7.14 refresh_tokens

id	UUID
user_id	UUID
token_hash	VARCHAR
expires_at	TIMESTAMP
revoked_at	TIMESTAMP NULL
created_at	TIMESTAMP

O token puro não deverá ser armazenado.

8. Relacionamentos principais

municipalities 1--N users
 municipalities 1--N occurrences
 municipalities 1--N departments

users 1--N occurrence_reports
 users 1--N occurrence_confirmations
 users 1--N repair_evaluations
 users 1--N notifications

occurrences 1--N occurrence_reports
 occurrences 1--N occurrence_images
 occurrences 1--N occurrence_confirmations
 occurrences 1--N occurrence_status_history
 occurrences 1--N repair_evaluations
 occurrences 1--N ai_analyses

9. Regras de negócio

RN-001 - Criação da ocorrência

Para registrar um problema, o sistema deverá exigir:

usuário autenticado;

imagem;

latitude;

longitude;

município;

descrição opcional.

Após o registro:

validar os dados;

armazenar a imagem;

consultar ocorrências próximas;

enviar os dados ao módulo de IA;

receber categoria, gravidade e possíveis duplicidades;

criar uma ocorrência ou vincular a uma existente;

criar o primeiro registro individual;

registrar histórico;

gerar notificação.

RN-002 - Consulta por proximidade

Antes de criar uma ocorrência, o sistema deverá procurar problemas:

em um raio inicial de 30 metros;

da mesma categoria ou categoria sugerida;

com status diferente de encerrado ou rejeitado;

registrados em período configurável.

Exemplo PostGIS:

```
ST_DWithin(
  location,
  ST_SetSRID(ST_MakePoint(:longitude, :latitude), 4326)::geography,
  30
)
```

O raio deverá ser configurável.

RN-003 - Duplicidade

A IA poderá sugerir duplicidade, mas o back-end será responsável pela decisão final.

A decisão deverá considerar:

proximidade;

categoria;

data;

similaridade informada pela IA;

status da ocorrência existente;

confirmação do usuário;

revisão humana, quando necessária.

RN-004 - Confirmação comunitária

Ao pressionar “Eu também vi”:

verificar se o usuário já confirmou;

criar a confirmação;

incrementar a contagem;

recalcular a prioridade;

criar notificação para o autor;

registrar auditoria.

RN-005 - Prioridade

A prioridade deverá ser calculada no back-end.

A IA poderá sugerir gravidade, mas não controlar diretamente a fila.

Fórmula inicial:

Prioridade =
 confirmações normalizadas x 0,35
 + gravidade x 0,25
 + tempo em aberto x 0,20
 + risco/reincidência x 0,20

A implementação deverá ser isolada em um serviço:

PriorityService

Isso permitirá alterar a fórmula sem modificar os controllers.

RN-006 - Alteração de status

Somente os perfis autorizados poderão alterar o status.

Transições válidas:

PENDING_REVIEW -> PUBLISHED

PENDING_REVIEW -> REJECTED

PUBLISHED -> FORWARDED

FORWARDED -> ACKNOWLEDGED

ACKNOWLEDGED -> UNDER_ANALYSIS

UNDER_ANALYSIS -> SCHEDULED

UNDER_ANALYSIS -> IN_PROGRESS

SCHEDULED -> IN_PROGRESS

IN_PROGRESS -> RESOLVED

RESOLVED -> CLOSED

RESOLVED -> CONTESTED

CONTESTED -> IN_PROGRESS

CONTESTED -> RESOLVED

Uma transição inválida deverá retornar erro HTTP 409 Conflict.

Toda alteração deverá:

validar permissão;

validar transição;

atualizar a ocorrência;

registrar histórico;

registrar auditoria;

gerar notificação.

RN-007 - Avaliação

Uma avaliação será permitida quando:

a ocorrência estiver como RESOLVED ou CLOSED;

o cidadão tiver criado ou confirmado a ocorrência;

o cidadão ainda não tiver avaliado.

Se uma proporção configurável de avaliações indicar que o problema não foi resolvido, o sistema deverá:

alterar para CONTESTED; ou

enviar para revisão administrativa.

RN-008 - Exclusão

Ocorrências não deverão ser apagadas fisicamente durante o uso normal.

Utilizar:

deleted_at

A exclusão deverá ser lógica, registrada em auditoria e permitida apenas para moderadores e administradores.

RN-009 - Anonimato público

O cidadão poderá optar por publicação anônima.

Mesmo nesse caso:

o back-end saberá quem criou o registro;
 o nome não será enviado nas rotas públicas;
 os dados pessoais continuarão protegidos.

RN-010 - Município do operador

Um operador municipal só poderá acessar e alterar ocorrências relacionadas ao município ao qual está vinculado.

10. Contrato com o módulo de IA

10.1 Requisição enviada pelo back-end

```
{
  "reportId": "uuid",
  "imageUrl": "https://storage.exemplo/imagem.jpg",
  "description": "Tem um buraco muito grande perto da escola",
  "latitude": -12.786,
  "longitude": -38.402,
  "nearbyOccurrences": [
    {
      "id": "uuid",
      "category": "POTHOLE",
      "distanceMeters": 12,
      "description": "Buraco na via principal",
      "imageUrls": [
        "https://storage.exemplo/imagem-anterior.jpg"
      ]
    }
  ]
}
```

10.2 Resposta esperada

```
{
  "category": "POTHOLE",
  "subcategory": "ASPHALT_DAMAGE",
  "severity": 4,
  "risk": "HIGH",
  "confidence": 0.91,
  "summary": "Buraco de grande dimensão próximo a uma escola.",
  "requiresHumanReview": false,
  "possibleDuplicates": [
    {
      "occurrenceId": "uuid",
      "similarity": 0.87,
      "reason": "Mesma categoria, localização próxima e imagem semelhante."
    }
  ]
}
```

10.3 Regras do contrato

o serviço de IA não acessará diretamente o banco;

o serviço de IA não alterará ocorrências;
toda resposta será validada por Zod;
respostas inválidas serão rejeitadas;
falha na IA não deverá derrubar a criação da ocorrência;
uma ocorrência poderá ficar como PENDING_REVIEW;
o resultado bruto deverá ser salvo em ai_analyses;
confiança baixa deverá gerar revisão humana.

11. Endpoints da API

Prefixo sugerido:

/api/v1

11.1 Autenticação

POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
GET /api/v1/auth/me
POST /api/v1/auth/change-password

11.2 Usuários

GET /api/v1/users/me
PATCH /api/v1/users/me
DELETE /api/v1/users/me

Rotas administrativas:

GET /api/v1/admin/users
GET /api/v1/admin/users/:id
PATCH /api/v1/admin/users/:id/status
PATCH /api/v1/admin/users/:id/role

11.3 Ocorrências públicas

GET /api/v1/occurrences
GET /api/v1/occurrences/:id
GET /api/v1/occurrences/nearby
GET /api/v1/occurrences/map

Filtros:

municipalityId
neighborhood
category
status
risk
priority
latitude
longitude
radius
startDate
endDate

page
limit

11.4 Criação e atualização

POST /api/v1/occurrences
 PATCH /api/v1/occurrences/:id
 POST /api/v1/occurrences/:id/images
 GET /api/v1/occurrences/:id/timeline

11.5 Confirmações

POST /api/v1/occurrences/:id/confirmations
 DELETE /api/v1/occurrences/:id/confirmations/me
 GET /api/v1/occurrences/:id/confirmations/count

11.6 Status

PATCH /api/v1/occurrences/:id/status
 GET /api/v1/occurrences/:id/status-history

Exemplo:

```
{
  "status": "IN_PROGRESS",
  "reason": "Equipe enviada ao local.",
  "publicMessage": "O reparo foi iniciado."
}
```

11.7 Avaliações

POST /api/v1/occurrences/:id/evaluations
 GET /api/v1/occurrences/:id/evaluations
 GET /api/v1/occurrences/:id/evaluations/summary
 PATCH /api/v1/occurrences/:id/evaluations/me

11.8 Notificações

GET /api/v1/notifications
 GET /api/v1/notifications/unread-count
 PATCH /api/v1/notifications/:id/read
 PATCH /api/v1/notifications/read-all

11.9 Painel municipal

GET /api/v1/dashboard/summary
 GET /api/v1/dashboard/by-category
 GET /api/v1/dashboard/by-neighborhood
 GET /api/v1/dashboard/by-status
 GET /api/v1/dashboard/priority-ranking
 GET /api/v1/dashboard/resolution-time
 GET /api/v1/dashboard/heatmap
 GET /api/v1/dashboard/export

Os endpoints deverão considerar:

município do operador;

período;

categoria;
 bairro;
 status.

11.10 Integração interna com IA

POST /api/v1/internal/ai/classify
 POST /api/v1/internal/ai/find-duplicates
 POST /api/v1/internal/ai/recalculate-priority

Essas rotas não deverão ficar abertas publicamente.

11.11 Webhooks e n8n

POST /api/v1/webhooks/telegram/report
 POST /api/v1/webhooks/whatsapp/report
 POST /api/v1/webhooks/status-update

Cada webhook deverá possuir:

segredo de autenticação;
 validação de assinatura;
 idempotência;
 logs;
 limite de requisições.

12. Respostas padronizadas

12.1 Sucesso

```
{
  "success": true,
  "data": {},
  "meta": {
    "requestId": "uuid"
  }
}
```

12.2 Erro

```
{
  "success": false,
  "error": {
    "code": "INVALID_STATUS_TRANSITION",
    "message": "Não é possível alterar diretamente de PUBLISHED para RESOLVED.",
    "details": {}
  },
  "meta": {
    "requestId": "uuid"
  }
}
```

13. Códigos HTTP

200 OK
201 Created
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
413 Payload Too Large
422 Unprocessable Entity
429 Too Many Requests
500 Internal Server Error
503 Service Unavailable

14. Validações

14.1 Usuário

e-mail válido;
senha com comprimento mínimo;
e-mail único;
telefone normalizado;
perfil não definido pelo próprio usuário;
município válido.

14.2 Ocorrência

latitude entre -90 e 90;
longitude entre -180 e 180;
descrição limitada;
imagem obrigatória na criação inicial;
formatos permitidos: JPEG, PNG e WebP;
tamanho máximo configurável;
município compatível com a localização;
categoria existente;
status válido.

14.3 Avaliação

nota entre 1 e 5;
usuário relacionado à ocorrência;
ocorrência resolvida;
apenas uma avaliação por usuário.

15. Segurança

15.1 Autenticação

senha armazenada somente como hash;
access token com curta duração;
refresh token revogável;
refresh token armazenado como hash;
logout revoga o token;
troca de senha revoga sessões anteriores.

15.2 Autorização

Criar middlewares:
authenticate
authorizeRoles
authorizeMunicipality
authorizeOccurrenceOwner

15.3 Proteção da API

Helmet;
rate limit;
CORS restrito;
limite de payload;
validação Zod;
SQL parametrizado;
tratamento centralizado de erros;
logs sem senhas ou tokens;
proteção contra upload malicioso.

15.4 Upload

validar MIME type real;
gerar nome aleatório;
não usar o nome original diretamente;
impedir arquivos executáveis;
limitar tamanho;
impedir acesso a buckets privados sem autorização.

15.5 LGPD

coletar somente dados necessários;
não expor e-mail ou telefone;
ocultar nome em publicação anônima;
permitir exclusão lógica da conta;

manter registros administrativos necessários;
evitar endereço residencial exato em respostas públicas;
armazenar consentimentos necessários.

16. Estrutura de pastas sugerida

```
src/
|-- app.ts
|-- server.ts
|-- config/
|   |-- env.ts
|   |-- database.ts
|   |-- storage.ts
|   `-- logger.ts
|-- database/
|   |-- migrations/
|   |-- seeds/
|   `-- schema/
|-- shared/
|   |-- errors/
|   |-- middleware/
|   |-- types/
|   |-- utils/
|   `-- validators/
|-- modules/
|   |-- auth/
|   |   |-- auth.controller.ts
|   |   |-- auth.service.ts
|   |   |-- auth.repository.ts
|   |   |-- auth.routes.ts
|   |   `-- auth.schemas.ts
|   |-- users/
|   |-- municipalities/
|   |-- categories/
|   |-- occurrences/
|   |-- confirmations/
|   |-- evaluations/
|   |-- notifications/
|   |-- dashboard/
|   |-- ai/
|   |-- storage/
|   `-- audit/
|-- docs/
|   `-- openapi.ts
`-- tests/
    |-- unit/
    |-- integration/
    `-- helpers/
```

Cada módulo deverá possuir, quando necessário:

```
controller
service
repository
routes
```

schemas
types
tests

17. Variáveis de ambiente

NODE_ENV=development
PORT=3333

DATABASE_URL=postgresql://user:password@localhost:5432/tanarua

JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d

CORS_ORIGIN=http://localhost:5173

STORAGE_PROVIDER=s3
STORAGE_BUCKET=
STORAGE_REGION=
STORAGE_ENDPOINT=
STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=

AI_SERVICE_URL=http://localhost:8000
AI_SERVICE_SECRET=

N8N_WEBHOOK_SECRET=

MAX_IMAGE_SIZE_MB=8
DUPLICATE_RADIUS_METERS=30
AI_MIN_CONFIDENCE=0.75

O projeto deverá possuir .env.example, sem segredos reais.

18. Dados fictícios para desenvolvimento

Criar seed com:

um município;
cinco bairros;
seis categorias;
um cidadão;
um operador municipal;
um moderador;
um administrador;
vinte ocorrências;
confirmações;
alterações de status;
avaliações;

notificações.

Contas fictícias poderão ser usadas somente no ambiente de desenvolvimento.

19. Testes obrigatórios

19.1 Autenticação

cadastro válido;

e-mail duplicado;

senha incorreta;

refresh token;

logout;

rota protegida;

perfil sem permissão.

19.2 Ocorrências

criação válida;

criação sem imagem;

coordenada inválida;

listagem com filtros;

consulta próxima;

usuário sem autenticação;

acesso a ocorrência inexistente.

19.3 Confirmações

primeira confirmação;

tentativa duplicada;

remoção da confirmação;

atualização da contagem;

recálculo da prioridade.

19.4 Status

transição válida;

transição inválida;

operador de outro município;

histórico gerado;

auditoria gerada;

notificação gerada.

19.5 Avaliação

avaliação válida;

avaliação duplicada;

usuário não relacionado;
ocorrência não resolvida;
avaliação negativa.

19.6 IA

resposta válida;
resposta inválida;
timeout;
confiança baixa;
serviço indisponível;
possível duplicidade.

20. Requisitos de qualidade

O código deverá:

- utilizar TypeScript estrito;
- não utilizar any sem justificativa;
- separar controllers, services e repositories;
- validar todas as entradas;
- possuir tratamento centralizado de erros;
- utilizar transações em operações críticas;
- possuir migrations;
- possuir testes;
- possuir Swagger;
- possuir README;
- utilizar paginação;
- evitar consultas N+1;
- registrar logs;
- não expor dados pessoais.

21. Critérios de aceite do MVP

O back-end estará pronto quando:

- o projeto iniciar localmente com um único comando;
- o banco puder ser criado por migration;
- os dados fictícios puderem ser inseridos por seed;
- usuários puderem se cadastrar e entrar;
- cidadãos puderem criar ocorrências com imagem e localização;
- ocorrências puderem ser exibidas no mapa;
- o sistema puder consultar problemas próximos;
- usuários puderem confirmar problemas;

- confirmações atualizarem a prioridade;
- operadores puderem alterar status;
- toda alteração gerar histórico;
- usuários puderem avaliar reparos;
- o módulo de IA puder ser chamado;
- falhas da IA forem tratadas;
- o painel receber indicadores;
- notificações internas funcionarem;
- permissões forem respeitadas;
- Swagger estiver disponível;
- testes essenciais forem aprovados;
- não existirem erros no build TypeScript.

22. Ordem de desenvolvimento

Fase 0 - Inicialização

- criar projeto;
- configurar TypeScript;
- configurar Express;
- configurar ESLint;
- configurar Prettier;
- configurar variáveis de ambiente;
- configurar Docker;
- criar rota /health.

Critério:

GET /health -> 200

Fase 1 - Banco de dados

- configurar PostgreSQL;
- habilitar PostGIS;
- criar migrations;
- criar tabelas;
- criar índices;
- criar seeds;
- configurar conexão.

Índices importantes:

- e-mail;
- telefone;
- protocolo;
- município;

categoria;
status;
data;
localização geográfica;
ocorrência e usuário nas confirmações.

Fase 2 - Autenticação

cadastro;
login;
JWT;
refresh token;
logout;
/auth/me;
hash de senha;
middlewares;
controle de perfis.

Fase 3 - Ocorrências

criação;
consulta;
paginação;
filtros;
localização;
upload;
página de detalhes;
ocorrências do usuário;
mapa público.

Fase 4 - Confirmações e prioridade

“Eu também vi”;
prevenção de duplicidade;
contagem;
recálculo de prioridade;
notificações;
testes.

Fase 5 - Status e histórico

máquina de estados;
alteração por operador;
histórico;
auditoria;

setor responsável;
previsão;
conclusão do reparo.

Fase 6 - Avaliações

avaliação;
resumo das avaliações;
contestação;
reabertura;
indicadores.

Fase 7 - Integração com IA

contrato;
cliente HTTP;
validação;
timeout;
retry limitado;
registro do resultado;
classificação;
duplicidades;
revisão humana.

Fase 8 - Dashboard

resumo;
categorias;
bairros;
status;
prioridade;
tempo de resolução;
mapa de calor;
exportação.

Fase 9 - Notificações e n8n

notificações internas;
eventos;
webhooks;
autenticação de webhook;
idempotência;
integração com o Integrante 5.

Fase 10 - Finalização

testes;
Swagger;
README;
Docker Compose;
revisão de segurança;
dados de demonstração;
deploy;
documentação da integração.

23. Definição de pronto

Uma tarefa somente será considerada concluída quando:

- o código estiver implementado;
- a migration necessária estiver criada;
- as entradas estiverem validadas;
- os erros estiverem tratados;
- os testes estiverem aprovados;
- a documentação estiver atualizada;
- o TypeScript compilar;
- a rota estiver documentada no Swagger;
- não houver segredos expostos;
- as permissões estiverem testadas.

24. Entregáveis do Integrante 2

Ao final, deverão ser entregues:

- repositório do back-end;
- banco modelado;
- migrations;
- seeds;
- API REST;
- autenticação;
- documentação Swagger;
- arquivo .env.example;
- Docker Compose;
- coleção de testes da API;
- README de instalação;
- contrato de integração com IA;
- contrato de integração com o painel;
- contrato de integração com n8n;

diagrama do banco;
relatório das regras de segurança.

Parte II - Adendo Técnico 1.1

Este adendo complementa o PRD Técnico 1.0. Em caso de conflito, as regras desta parte prevalecem sobre a versão 1.0.

25. Decisões técnicas definitivas

Para impedir alterações de ferramentas e padrões durante o desenvolvimento, ficam estabelecidas as seguintes escolhas:

Runtime: Node.js
Linguagem: TypeScript em modo estrito
Framework HTTP: Express
Banco de dados: PostgreSQL
Extensão geográfica: PostGIS
ORM e query builder: Drizzle ORM
Migrations: Drizzle Kit
Validação: Zod
Hash de senha: Argon2id
Access token: JWT
Refresh token: token opaco aleatório
Documentação: OpenAPI 3 + Swagger UI
Testes: Vitest + Supertest
Logs: Pino
Lint: ESLint
Formatação: Prettier
Containerização: Docker Compose
Storage: serviço compatível com a API S3

Não será utilizado Knex neste projeto.

O Drizzle ORM foi escolhido por:

- possuir boa integração com TypeScript;
- permitir definição tipada do schema;
- permitir migrations versionadas;
- oferecer consultas SQL próximas do PostgreSQL;
- permitir execução de SQL parametrizado para PostGIS;
- reduzir abstrações excessivas sobre consultas geográficas;
- facilitar a inferência de tipos das entidades.

Para consultas geográficas complexas, será permitido utilizar SQL parametrizado por meio do próprio Drizzle.

26. Convenções gerais

26.1 Identificadores

As entidades utilizarão UUID. Os UUIDs serão gerados pelo back-end ou pelo PostgreSQL. Nenhum identificador sequencial interno deverá ser exposto como chave principal pública.

26.2 Datas e horários

Todos os horários deverão ser:

armazenados em UTC;

transmitidos em ISO 8601;

convertidos para o fuso do usuário somente na interface.

Fuso de referência:

America/Bahia

No PostgreSQL, utilizar preferencialmente TIMESTAMPTZ.

26.3 Nomes

Banco de dados: snake_case

TypeScript: camelCase

Enums e constantes: UPPER_SNAKE_CASE

26.4 API

Prefixo: /api/v1

Formato JSON: application/json

Upload: multipart/form-data

26.5 Paginação

Parâmetros:

page

limit

Valores:

page padrão: 1

limit padrão: 20

limit máximo: 100

Resposta:

```
{
  "success": true,
  "data": [],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 100,
    "totalPages": 5,
    "requestId": "uuid"
  }
}
```

27. Autenticação e sessões

27.1 Access token

O access token será um JWT enviado no cabeçalho:

Authorization: Bearer <access_token>

Duração padrão: 15 minutos.

Payload mínimo:

```
{
  "sub": "user-uuid",
  "role": "CITIZEN",
  "municipalityId": "municipality-uuid",
  "sessionId": "session-uuid"
}
```

Não incluir senha, telefone, endereço ou informações pessoais desnecessárias.

27.2 Refresh token

O refresh token será:

opaco;

aleatório;

criptograficamente seguro;

enviado por cookie httpOnly;

armazenado somente como hash no banco;

revogável;

associado a uma sessão.

Configuração do cookie:

httpOnly: true

secure: true em produção

sameSite: lax

path: /api/v1/auth

maxAge: 7 dias

Em desenvolvimento local, secure poderá ser falso.

27.3 Rotação

Cada utilização válida do refresh token deverá:

validar o hash armazenado;

verificar expiração e revogação;

revogar o token anterior;

gerar um novo refresh token;

registrar o novo hash;

emitir um novo access token.

A reutilização de um refresh token já revogado deverá invalidar a sessão relacionada.

27.4 Alteração de senha

Após a alteração da senha:

todas as sessões anteriores deverão ser revogadas;

um novo login deverá ser exigido;

a alteração deverá gerar registro de auditoria;

nenhum hash antigo deverá ser mantido.

28. Alterações na modelagem do banco

28.1 Tabela neighborhoods

neighborhoods

```
id          UUID
municipality_id UUID
name        VARCHAR
slug        VARCHAR
active      BOOLEAN
created_at  TIMESTAMPTZ
updated_at  TIMESTAMPTZ
```

Restrições:

FOREIGN KEY municipality_id REFERENCES municipalities(id)
 UNIQUE (municipality_id, slug)

Índices: municipality_id, name e slug.

A tabela será utilizada em cadastro de ocorrências, filtros, dashboard, mapa, indicadores municipais e seeds.

28.2 Alterações em categories

Adicionar:

code VARCHAR UNIQUE

O campo code será o identificador estável utilizado pela API e pela IA.

Categorias iniciais:

```
POTHOLE
PUBLIC_LIGHTING
WATER_LEAK
SEWAGE
GARBAGE
SIDEWALK_DAMAGE
FLOODING
FALLEN_TREE
TRAFFIC_SIGN
TRAFFIC_LIGHT
OTHER
```

O campo name será utilizado somente para exibição.

```
{
  "code": "POTHOLE",
  "name": "Buraco na via",
  "slug": "buraco-na-via"
}
```

28.3 Alterações em occurrences

```
id          UUID
protocol    VARCHAR UNIQUE
title       VARCHAR
description  TEXT NULL
category_id UUID NULL
municipality_id UUID
```

neighborhood_id	UUID NULL
created_by	UUID
assigned_department_id	UUID NULL
assigned_by	UUID NULL
assigned_at	TIMESTAMPTZ NULL
status	ENUM
severity	INTEGER NULL
priority_score	DECIMAL DEFAULT 0
risk_level	ENUM NULL
address	VARCHAR NULL
neighborhood_text	VARCHAR NULL
latitude	DECIMAL
longitude	DECIMAL
location	GEOGRAPHY(Point, 4326)
location_accuracy	DECIMAL NULL
anonymous_publication	BOOLEAN DEFAULT FALSE
confirmation_count	INTEGER DEFAULT 0
duplicate_of_occurrence_id	UUID NULL
expected_resolution_at	TIMESTAMPTZ NULL
scheduled_for	TIMESTAMPTZ NULL
resolution_description	TEXT NULL
resolved_at	TIMESTAMPTZ NULL
resolved_by	UUID NULL
closed_at	TIMESTAMPTZ NULL
closed_by	UUID NULL
first_reported_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ
deleted_at	TIMESTAMPTZ NULL

Alterações:

category_id, severity e risk_level poderão ser nulos enquanto a ocorrência aguarda classificação;

citizen_count será removido do MVP;

confirmation_count será mantido como contador transacional;

neighborhood_id representará o bairro cadastrado;

neighborhood_text armazenará o bairro informado originalmente, quando necessário;

duplicate_of_occurrence_id indicará a ocorrência principal;

serão adicionados campos do atendimento municipal.

Restrições:

latitude BETWEEN -90 AND 90

longitude BETWEEN -180 AND 180

severity BETWEEN 1 AND 5, quando não nulo

priority_score BETWEEN 0 AND 100

confirmation_count >= 0

duplicate_of_occurrence_id não poderá apontar para a própria ocorrência.

28.4 Relações entre ocorrências

Poderá ser criada a tabela:

occurrence_relations

id	UUID
----	------

source_occurrence_id UUID
 target_occurrence_id UUID
 relation_type ENUM
 created_by UUID
 created_at TIMESTAMPTZ

Tipos:

DUPLICATE_OF
 RELATED_TO
 MERGED_INTO
 RECURRENCE_OF

Restrições:

source_occurrence_id <> target_occurrence_id
 UNIQUE (source_occurrence_id, target_occurrence_id, relation_type)

Para o MVP, duplicate_of_occurrence_id será suficiente. A tabela será opcional para fusão e separação avançada.

28.5 Alterações em ai_analyses

Adicionar:

suggested_subcategory VARCHAR NULL

Restrições:

confidence BETWEEN 0 AND 1
 suggested_severity BETWEEN 1 AND 5, quando não nulo

O campo raw_result deverá armazenar o resultado bruto validado ou o resultado recebido antes da validação, com indicação do estado da análise.

Nunca armazenar segredos, tokens, cabeçalhos de autenticação ou dados pessoais desnecessários.

28.6 Alterações em repair_evaluations

UNIQUE (occurrence_id, user_id)
 rating BETWEEN 1 AND 5
 service_quality BETWEEN 1 AND 5, quando não nulo

28.7 Alterações em refresh_tokens

id UUID
 session_id UUID
 user_id UUID
 token_hash VARCHAR UNIQUE
 expires_at TIMESTAMPTZ
 revoked_at TIMESTAMPTZ NULL
 last_used_at TIMESTAMPTZ NULL
 ip_address VARCHAR NULL
 user_agent TEXT NULL
 created_at TIMESTAMPTZ

O token em texto puro nunca deverá ser armazenado.

28.8 Tabela webhook_events

webhook_events

id UUID

```

provider          VARCHAR
external_event_id VARCHAR
event_type        VARCHAR
payload_hash      VARCHAR
status            ENUM
response_code      INTEGER NULL
error_message      TEXT NULL
received_at        TIMESTAMPTZ
processed_at       TIMESTAMPTZ NULL

```

Status:

```

RECEIVED
PROCESSING
PROCESSED
FAILED
IGNORED

```

Restrição:

UNIQUE (provider, external_event_id)

O mesmo evento externo não poderá gerar duas ocorrências.

28.9 Tabela outbox_events

Adicionar na fase de integrações:

outbox_events

```

id          UUID
event_type   VARCHAR
entity_type  VARCHAR
entity_id    UUID
payload      JSONB
status       ENUM
attempts     INTEGER DEFAULT 0
available_at TIMESTAMPTZ
processed_at TIMESTAMPTZ NULL
last_error   TEXT NULL
created_at   TIMESTAMPTZ

```

Status:

```

PENDING
PROCESSING
PROCESSED
FAILED

```

Essa tabela permitirá processar notificações, chamadas ao n8n, Telegram, WhatsApp e demais integrações. A criação do evento deverá ocorrer na mesma transação da alteração principal.

29. Geração do protocolo

Formato:

TNR-AAAA-NNNNNN

Exemplo:

TNR-2026-000001

Regras:

gerado exclusivamente pelo back-end;
 sequência independente por ano;
 nunca reutilizado;
 protegido contra concorrência;
 não derivado diretamente do UUID;
 imutável após a criação.

Tabela:

protocol_counters

```
year      INTEGER PRIMARY KEY
last_value BIGINT
updated_at TIMESTAMPTZ
```

A obtenção do próximo número deverá ocorrer com bloqueio ou operação atômica dentro de transação.

30. Máquina de estados definitiva

30.1 Transições operacionais

PENDING_REVIEW -> PUBLISHED
 PENDING_REVIEW -> REJECTED
 PENDING_REVIEW -> DUPLICATE

PUBLISHED -> FORWARDED
 PUBLISHED -> REJECTED
 PUBLISHED -> DUPLICATE

FORWARDED -> ACKNOWLEDGED
 FORWARDED -> UNDER_ANALYSIS

ACKNOWLEDGED -> UNDER_ANALYSIS

UNDER_ANALYSIS -> SCHEDULED
 UNDER_ANALYSIS -> IN_PROGRESS
 UNDER_ANALYSIS -> REJECTED

SCHEDULED -> IN_PROGRESS
 SCHEDULED -> UNDER_ANALYSIS

IN_PROGRESS -> RESOLVED
 IN_PROGRESS -> UNDER_ANALYSIS

RESOLVED -> CLOSED
 RESOLVED -> CONTESTED
 RESOLVED -> IN_PROGRESS

CLOSED -> CONTESTED

CONTESTED -> IN_PROGRESS
 CONTESTED -> RESOLVED

REJECTED -> PENDING_REVIEW

DUPLICATE -> PENDING_REVIEW

30.2 Permissões

CITIZEN

Não poderá alterar diretamente o status. Poderá contestar por meio de avaliação, solicitar revisão pelos mecanismos previstos e acompanhar o histórico público.

CITY_OPERATOR

Poderá executar transições operacionais relacionadas ao atendimento, somente no seu município.

Não poderá excluir definitivamente, alterar município, marcar como DUPLICATE, reclassificar usuário ou alterar logs.

MODERATOR

Poderá publicar, rejeitar, marcar como duplicada, reabrir ocorrência rejeitada, desfazer duplicidade, corrigir categoria e revisar resultados da IA.

ADMIN

Poderá executar todas as transições válidas. Mesmo o administrador não poderá executar transição inexistente sem procedimento administrativo específico e auditado.

30.3 Operação obrigatória

Toda alteração de status deverá ocorrer em uma transação contendo:

bloqueio ou leitura consistente da ocorrência;

validação da existência;

validação do status atual;

validação da transição;

validação da permissão;

validação do município;

atualização da ocorrência;

inserção no histórico;

inserção no log de auditoria;

criação de notificações ou eventos;

commit.

Uma transição inválida retornará 409 Conflict, código INVALID_STATUS_TRANSITION.

31. Configurações iniciais do MVP

Raio de duplicidade: 30 metros

Período para duplicidade: 30 dias

Confiança mínima da IA: 0,75

Similaridade mínima sugerida: 0,80

Vinculação automática por IA: desativada

Prazo para editar avaliação: 7 dias

Percentual de avaliações negativas para contestação: 50%

Quantidade mínima de avaliações para contestação: 3

Máximo de imagens por ocorrência: 5
 Tamanho máximo por imagem: 8 MB
 Comprimento máximo do título: 150 caracteres
 Comprimento máximo da descrição: 2.000 caracteres
 Comentário de confirmação: 500 caracteres
 Paginação padrão: 20 registros
 Paginação máxima: 100 registros
 Timeout da IA: 8 segundos
 Máximo de tentativas da IA: 2

Variáveis:

DUPLICATE_RADIUS_METERS=30
 DUPLICATE_PERIOD_DAYS=30
 AI_MIN_CONFIDENCE=0.75
 AI_DUPLICATE_MIN_SIMILARITY=0.80
 AI_TIMEOUT_MS=8000
 AI_MAX_ATTEMPTS=2
 EVALUATION_EDIT_DAYS=7
 CONTESTATION_NEGATIVE_PERCENTAGE=50
 CONTESTATION_MIN_EVALUATIONS=3
 MAX_IMAGES_PER_OCCURRENCE=5
 MAX_IMAGE_SIZE_MB=8
 DEFAULT_PAGE_SIZE=20
 MAX_PAGE_SIZE=100

Nenhuma ocorrência será unida automaticamente apenas pela decisão da IA no MVP.

32. Fórmula de prioridade definitiva

A prioridade terá valor entre 0 e 100.

32.1 Confirmações

$\text{confirmationScore} = \min(\text{confirmationCount} / 20, 1) \times 100$

32.2 Gravidade

$\text{severityScore} = \text{severity} / 5 \times 100$

Quando a gravidade não estiver definida: $\text{severityScore} = 0$.

32.3 Tempo em aberto

$\text{ageScore} = \min(\text{daysOpen} / 30, 1) \times 100$

32.4 Risco

LOW = 25
 MEDIUM = 50
 HIGH = 75
 CRITICAL = 100

Quando o risco não estiver definido: $\text{riskScore} = 0$.

32.5 Cálculo

$\text{priorityScore} =$
 $\text{confirmationScore} \times 0,35$

+ severityScore x 0,25
 + ageScore x 0,20
 + riskScore x 0,20

O resultado será limitado entre 0 e 100, arredondado para duas casas e calculado exclusivamente pelo PriorityService.

O recálculo será executado quando:

uma confirmação for adicionada ou removida;
 gravidade ou risco forem alterados;
 uma análise da IA for aprovada;
 uma ocorrência for reaberta;
 uma rotina administrativa for executada.

O fator de tempo poderá ser recalculado por tarefa agendada ou no momento da consulta ordenada por prioridade.

33. Contadores

confirmation_count representará a quantidade de registros ativos em occurrence_confirmations.

Regras:

inclusão e incremento na mesma transação;
 remoção e decremento na mesma transação;
 nunca permitir valor negativo;
 disponibilizar rotina administrativa para recálculo;
 considerar a tabela de confirmações como fonte oficial.

O número de cidadãos relacionados será calculado por consulta utilizando usuários únicos entre occurrence_reports e occurrence_confirmations. Não será armazenado no MVP.

34. Privacidade da localização

O banco armazenará a coordenada original necessária para agrupamento, atendimento municipal, auditoria e análise geográfica.

34.1 Resposta pública

Nas rotas públicas:

latitude e longitude serão arredondadas para quatro casas decimais;
 número residencial será removido;
 complemento não será exibido;
 nome, telefone e e-mail do cidadão não serão retornados;
 publicações anônimas não retornarão dados do autor.

34.2 Resposta administrativa

Operadores autorizados poderão visualizar a localização completa somente quando autenticados, vinculados ao município da ocorrência e quando a localização for necessária para atendimento. Moderadores e administradores seguirão suas permissões.

35. Fluxo definitivo de criação de ocorrência

autenticar o cidadão;
 validar campos com Zod;
 validar arquivo;
 validar localização;
 validar município;
 armazenar imagem;
 procurar ocorrências próximas;
 iniciar transação;
 gerar protocolo;
 criar ocorrência provisória como PENDING_REVIEW;
 criar primeiro occurrence_report;
 criar registro da imagem;
 criar histórico inicial;
 criar log de auditoria;
 confirmar a transação;
 chamar o serviço de IA;
 validar a resposta;
 salvar ai_analyses;
 atualizar categoria, gravidade, risco e resumo, quando permitido;
 sugerir duplicidades;
 publicar automaticamente somente quando as regras de confiança permitirem;
 manter em revisão quando houver erro, baixa confiança ou inconsistência;
 gerar notificação.

A indisponibilidade da IA não deverá remover nem invalidar a ocorrência já registrada.

Se o upload for concluído e a transação falhar, o back-end deverá tentar remover o arquivo órfão ou registrá-lo para limpeza posterior.

36. Decisão de duplicidade

A IA não tomará a decisão final. O serviço considerará:

distância
 categoria
 período
 similaridade da IA
 status da ocorrência existente
 decisão do cidadão
 decisão do moderador

Resultados possíveis:

CREATE_NEW
 LINK_TO_EXISTING
 REQUIRES_REVIEW

No MVP, uma ocorrência poderá ser vinculada diretamente quando o cidadão selecionar uma ocorrência existente ou um moderador confirmar a duplicidade. Apenas a pontuação da IA não poderá realizar fusão automática.

Ao marcar uma ocorrência como duplicada:

definir status = DUPLICATE;
preencher duplicate_of_occurrence_id;
manter reports e imagens;
registrar histórico e auditoria;
recalcular cidadãos relacionados da principal;
notificar usuários envolvidos.

37. Rotas e ordem no Express

Rotas específicas deverão ser registradas antes das parametrizadas:

GET /api/v1/occurrences/nearby
GET /api/v1/occurrences/map
GET /api/v1/occurrences/mine
GET /api/v1/occurrences/:id

O parâmetro :id deverá ser validado como UUID. A validação não poderá depender apenas da ordem das rotas.

38. Auditoria

Deverão gerar auditoria:

cadastro e alteração de usuário administrativo;
bloqueio e desbloqueio;
alteração de perfil;
login administrativo relevante;
troca de senha;
revogação de sessão;
criação e alteração de ocorrência;
exclusão lógica;
confirmação e remoção de confirmação;
alteração de status;
atribuição a departamento;
alteração de prazo;
conclusão;
contestação;
avaliação;
decisão de duplicidade;
revisão de IA;
processamento de webhook.

Campos sensíveis deverão ser removidos de previous_data e new_data.

Nunca registrar senha, hash de senha, access token, refresh token, hash de refresh token, segredo de webhook ou chave de storage.

39. Escopo definitivo do MVP

39.1 Obrigatório

Fase 0 até Fase 6
Banco PostgreSQL e PostGIS
Autenticação e sessões
Perfis e permissões
Isolamento por município
Ocorrências
Upload de imagens
Consulta por localização
Mapa
Confirmações
Prioridade
Status
Histórico
Auditoria
Departamentos
Avaliações
Notificações internas básicas
Swagger
Migrations
Seeds
Testes essenciais
Docker Compose
README

39.2 Integração demonstrativa

Serviço de IA real ou mockado
Sugestão de categoria e gravidade
Sugestão de duplicidade
Um webhook funcional para n8n
Dashboard com resumo
Dashboard por categoria
Dashboard por status
Ranking de prioridade

39.3 Funcionalidades adicionais

WhatsApp completo
Telegram completo
Mapa de calor avançado
Exportações avançadas
Fila distribuída
Fusão e separação complexa
Contestação automática avançada
Gestão nacional de municípios
Notificações push externas
Rotinas avançadas de limpeza de storage

Funcionalidades adicionais não poderão impedir a entrega do núcleo obrigatório.

40. Documentos complementares obrigatórios

README.md
.env.example
docker-compose.yml
OpenAPI/Swagger
Diagrama ER
Contrato da IA
Contrato do painel
Contrato do n8n
Relatório de segurança
Coleção de requisições ou arquivo HTTP

41. Regra de evolução do projeto

Cada fase deverá respeitar contratos implementados anteriormente.

Uma alteração de contrato deverá informar:

motivo;
arquivos afetados;
endpoints afetados;
impacto no banco;
migration necessária;
impacto nos testes;
impacto no aplicativo, painel, IA ou n8n;
procedimento de compatibilidade ou migração.

Nenhuma migration já executada deverá ser editada retroativamente. Correções deverão ser implementadas em nova migration.

42. Critério adicional de conclusão

Uma fase não estará pronta somente porque o código foi escrito. Será necessário comprovar:

instalação das dependências
subida dos containers
execução das migrations
execução dos seeds, quando aplicável
lint sem erros
testes sem erros
build sem erros
rotas validadas
documentação atualizada
ausência de segredos
ausência de funções vazias
ausência de TODO usado como implementação

Quando alguma parte não puder ser concluída, a limitação deverá ser declarada claramente.

Parte III - Adendo Técnico 1.2: Interfaces de desenvolvimento e validação

Este documento complementa o PRD Técnico 1.0 e o Adendo Técnico 1.1.

43. Interfaces de desenvolvimento

O back-end deverá fornecer interfaces que permitam utilização, inspeção e validação sem depender do front-end. O desenvolvedor deverá conseguir testar todas as funcionalidades por meio de:

Swagger UI;

Adminer;

coleção Bruno ou Postman;

comandos curl;

testes automatizados;

dados fictícios criados por seeds.

O aplicativo do cidadão e o painel municipal não serão necessários para validar o back-end. Essas interfaces não substituem os testes automatizados.

44. Serviços locais obrigatórios

API: `http://localhost:3333`

Swagger: `http://localhost:3333/docs`

Adminer: `http://localhost:8080`

PostgreSQL: `localhost:5432`

O Docker Compose deverá iniciar PostgreSQL com PostGIS, Adminer, volumes persistentes e healthcheck do PostgreSQL. A API poderá ser executada no ambiente Node.js durante o desenvolvimento e adicionada ao Docker Compose posteriormente.

45. Swagger UI

O projeto deverá disponibilizar:

GET `/docs`

GET `/docs/openapi.json`

A documentação deverá utilizar OpenAPI 3.0 ou superior.

45.1 Disponibilidade progressiva

O Swagger será criado na Fase 0 e atualizado em todas as fases seguintes. Na Fase 0 deverá documentar GET `/health` e GET `/health/database`.

Cada nova rota deverá ser incluída na mesma fase em que for implementada. Uma rota ausente do OpenAPI não será considerada concluída.

46. Conteúdo obrigatório da documentação OpenAPI

Cada endpoint deverá documentar:

método e caminho;

resumo e descrição;
 módulo;
 autenticação e perfis;
 parâmetros de rota e consulta;
 cabeçalhos;
 corpo e tipo de conteúdo;
 exemplos de requisição e resposta;
 códigos HTTP;
 erros de validação, autenticação, autorização e negócio.
 Os exemplos deverão ser compatíveis com os schemas Zod e o banco.

47. Autorização JWT no Swagger

A especificação deverá possuir o esquema:

```
type: http
scheme: bearer
bearerFormat: JWT
```

O Swagger UI deverá apresentar o botão **Authorize** e enviar o access token em:

Authorization: Bearer <access_token>

Na Fase 0 será preparada a estrutura. Na Fase 2 será ativado o fluxo real, com login, refresh, logout, troca de senha e rotas protegidas. O refresh token httpOnly não deverá ser copiado para o campo JWT.

48. Upload pelo Swagger

Rotas de upload deverão usar multipart/form-data e permitir seleção de arquivos no Swagger.

Exemplo:

```
type: object
required:
  - image
properties:
  image:
    type: string
    format: binary
  description:
    type: string
  latitude:
    type: number
  longitude:
    type: number
```

A documentação deverá informar formatos, tamanho, quantidade, campos e erros.

Códigos esperados:

```
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden
413 Payload Too Large
```

415 Unsupported Media Type

422 Unprocessable Entity

O código 415 Unsupported Media Type passa a integrar a lista oficial do PRD.

49. Adminer

O Docker Compose deverá incluir Adminer.

URL: `http://localhost:8080`

Sistema: PostgreSQL

Servidor: postgres

Porta: 5432

Banco: tanarua

Usuário: definido no .env

Senha: definida no .env

No navegador, o servidor deverá ser o nome do serviço Docker postgres, e não localhost. Nenhuma credencial real ficará fixa no docker-compose.yml.

50. Seeds de desenvolvimento

O comando será:

`npm run db:seed`

Os seeds deverão ser reproduzíveis, seguros para desenvolvimento e bloqueados em produção.

50.1 Usuários obrigatórios

Criar usuários fictícios para:

CITIZEN

CITY_OPERATOR

MODERATOR

ADMIN

Também criar, quando necessário:

cidadão e operador de outro município;

usuário bloqueado;

usuário com sessão revogada;

usuário relacionado a uma ocorrência;

usuário não relacionado.

Senhas fictícias existirão somente em desenvolvimento, serão documentadas no README e armazenadas como hash.

50.2 Dados obrigatórios

Os seeds deverão criar progressivamente municípios, bairros, categorias, departamentos, usuários, ocorrências, reports, imagens fictícias ou metadados, confirmações, históricos, avaliações, notificações, análises de IA e eventos de webhook.

50.3 Idempotência

Executar o seed mais de uma vez não deverá gerar duplicações descontroladas. A estratégia deverá ser documentada.

51. Coleção de API

O projeto deverá possuir coleção Bruno ou Postman, versionada no repositório. A escolha deverá ser mantida.

Estrutura sugerida:

```
api-client/
|-- environments/
|   |-- local
|-- auth/
|-- users/
|-- occurrences/
|-- confirmations/
|-- status/
|-- evaluations/
|-- notifications/
|-- dashboard/
|-- ai/
`-- webhooks/
```

51.1 Variáveis

```
baseUrl
accessToken
citizenAccessToken
operatorAccessToken
moderatorAccessToken
adminAccessToken
municipalityId
occurrenceId
userId
```

Nenhum token real será versionado.

51.2 Atualização por fase

Cada fase deverá incluir novas rotas, exemplos válidos e inválidos, autenticação e códigos esperados. Uma rota deverá ser testável pelo Swagger ou pela coleção.

52. Scripts npm obrigatórios

```
npm run dev
npm run build
npm run lint
npm run test
npm run test:unit
npm run test:integration
npm run test:coverage
npm run db:migrate
npm run db:seed
npm run validate
```

Scripts adicionais permitidos:

```
npm run lint:fix
npm run format
npm run format:check
npm run test:watch
```

```
npm run db:generate
npm run db:studio
npm run docker:up
npm run docker:down
```

52.1 Responsabilidade dos scripts

dev: API em desenvolvimento com reinicialização automática;

build: compilação de produção;

lint: ESLint sem correção automática;

test: suíte padrão;

test:unit: testes unitários;

test:integration: testes de integração;

test:coverage: relatório de cobertura;

db:migrate: migrations pendentes;

db:seed: dados fictícios;

validate: lint, testes e build.

Implementação equivalente:

```
{
  "scripts": {
    "validate": "npm run lint && npm run test && npm run build"
  }
}
```

Qualquer falha deverá encerrar com código diferente de zero.

53. Testes automatizados por fase

Todas as fases deverão incluir testes relacionados às funcionalidades implementadas. Não será permitido deixar todos os testes para a Fase 10.

Quando aplicável, testar:

casos válidos e inválidos;

validação;

autenticação e autorização;

isolamento municipal;

conflitos de negócio;

indisponibilidade de dependências;

respostas padronizadas.

53.1 Testes de permissão

Toda rota protegida deverá testar:

usuário sem token

token inválido

token expirado

perfil permitido

perfil não permitido

usuário bloqueado

operador do município correto
 operador de outro município
 moderador
 administrador

53.2 Cobertura

npm run test:coverage

Metas iniciais:

Statements: 80%

Functions: 80%

Lines: 80%

Branches: 70%

Módulos críticos: autenticação, autorização, isolamento municipal, máquina de estados, prioridade, duplicidade, avaliações e webhooks.

54. Roteiro de teste manual no README

O README deverá permitir execução sem conhecimento prévio do código e incluir:

pré-requisitos e versão Node;

instalação;

criação do .env;

Docker Compose e healthcheck;

migrations e seeds;

inicialização da API;

Swagger e Adminer;

autenticação e obtenção do token;

botão Authorize;

execução das rotas;

conferência no Adminer;

coleção Bruno/Postman;

testes e cobertura;

npm run validate;

solução de problemas.

Roteiro mínimo da Fase 0:

npm install

`cp .env.example .env`

`docker compose up -d`

`npm run db:migrate`

`npm run lint`

`npm run test`

`npm run build`

`npm run dev`

URLs:

`http://localhost:3333/health`

`http://localhost:3333/health/database`

http://localhost:3333/docs
http://localhost:8080

55. Validação obrigatória em cada fase

Ao final de cada fase, a resposta da IA deverá conter a seção **Roteiro de validação da fase**.

55.1 Testes automatizados

Informar testes, arquivos, comandos, resultados esperados e dependências.

55.2 Teste pelo Swagger

Informar URL, endpoint, campos, perfil, token, corpo, resposta e código esperado.

55.3 Verificação no Adminer

Informar tabela, filtros, registros, campos e efeito esperado.

55.4 Casos de sucesso

Informar entrada, ação, resposta, código, alteração no banco, auditoria e notificação.

55.5 Casos de erro

Testar quando aplicável:

- dados ausentes
- dados inválidos
- recurso inexistente
- duplicidade
- estado incompatível
- arquivo inválido
- arquivo grande demais
- banco indisponível
- IA indisponível
- token ausente
- token inválido
- token expirado
- perfil sem permissão
- município incompatível

55.6 Testes de permissão

Apresentar matriz com rota, CITIZEN, CITY_OPERATOR, MODERATOR, ADMIN e resultado esperado.

Usar classificações:

- Permitido
- Negado com 401
- Negado com 403
- Restrito ao município
- Restrito ao proprietário

55.7 Resultado esperado

Informar o que comprova funcionamento, tabelas modificadas, logs, respostas e limitações.

56. Códigos HTTP nos roteiros

200 OK
 201 Created
 204 No Content
 400 Bad Request
 401 Unauthorized
 403 Forbidden
 404 Not Found
 409 Conflict
 413 Payload Too Large
 415 Unsupported Media Type
 422 Unprocessable Entity
 429 Too Many Requests
 500 Internal Server Error
 503 Service Unavailable

A resposta deverá corresponder ao OpenAPI.

57. Alterações na Fase 0

A Fase 0 incluirá adicionalmente:

Swagger UI em /docs;
 OpenAPI inicial e /docs/openapi.json;
 esquema JWT Bearer preparado;
 exemplos de health;
 Adminer no Docker Compose;
 scripts de testes separados e cobertura;
 db:seed inicial limitado;
 script validate;
 roteiro manual no README;
 estrutura inicial da coleção.

Serviços:

API: http://localhost:3333
 Swagger: http://localhost:3333/docs
 Adminer: http://localhost:8080
 PostgreSQL: localhost:5432

O seed da Fase 0 poderá criar somente o mínimo existente e não poderá fingir que tabelas futuras já foram criadas.

58. Alterações na Fase 1

A Fase 1 deverá criar tabelas definitivas, categorias, municípios, bairros, departamentos, preparar usuários fictícios, atualizar seed e documentar as verificações no Adminer. Senhas e sessões não serão simuladas como funcionais antes da autenticação.

59. Alterações na Fase 2

A Fase 2 deverá criar usuários de todos os perfis, ativar o botão Authorize, documentar autenticação, atualizar coleção e README, testar permissões e isolamento, e mostrar no Adminer usuários, sessões e refresh tokens em hash. Tokens puros não aparecerão no banco.

60. Alterações na Fase 3

A Fase 3 deverá documentar multipart/form-data, permitir upload no Swagger, incluir arquivo de exemplo, documentar limites, atualizar coleção, mostrar ocorrências/reports/imagens no Adminer, testar MIME/tamanho e validar privacidade.

61. Alterações nas fases seguintes

Cada fase deverá atualizar simultaneamente:

- código
- testes
- Swagger
- coleção de API
- README
- migrations
- seeds, quando aplicável
- roteiro de teste manual
- orientações do Adminer
- matriz de permissões

62. Critérios adicionais de pronto

Uma fase somente será concluída quando:

- rotas e schemas estiverem implementados;
- Swagger, exemplos e coleção estiverem atualizados;
- testes automatizados e de permissão forem aprovados;
- roteiro Swagger/Adminer e códigos HTTP estiverem documentados;
- README estiver atualizado;
- npm run validate terminar sem erros;
- não houver segredos, funções vazias ou TODO como implementação.

63. Comando final de validação

- npm run lint
- npm run test:unit
- npm run test:integration
- npm run test:coverage
- npm run build
- npm run validate

Quando houver alteração de banco:

- npm run db:migrate
- npm run db:seed

Também verificar Swagger UI, Adminer, coleção, curl, logs e healthcheck. A IA deverá informar honestamente o que não foi executado ou comprovado.

Parte IV - Checklist mestre de acompanhamento

64. Regra geral de acompanhamento

A IA deverá criar e manter este checklist atualizado antes de qualquer implementação.

Marcações:

- [] Não iniciado
- [~] Em desenvolvimento
- [x] Concluído e validado
- [!] Bloqueado
- [-] Fora do escopo da fase atual

Um item somente poderá ser marcado como [x] quando houver evidência de implementação, compilação, lint, testes, Swagger, coleção, roteiro manual, permissões, migrations e conferência no banco, conforme aplicável.

Apenas gerar código não permite marcar o item como concluído. Quando um comando não puder ser executado, o item ficará [~] ou [!] até o usuário enviar o resultado.

65. Evidências obrigatórias

Para cada item concluído, informar evidência objetiva, por exemplo:

- Evidência: npm run build executado sem erros.
- Evidência: teste GET /health aprovado.
- Evidência: tabela users conferida no Adminer.
- Evidência: rota documentada em /docs.
- Evidência: migration executada.

Quando depender do usuário:

Evidência pendente: aguardando resultado do comando npm run validate.

A IA não poderá inventar resultados.

66. Resumo geral das fases

- ☐ Fase 0 - Inicialização
- ☐ Fase 1 - Banco de dados
- ☐ Fase 2 - Autenticação e usuários
- ☐ Fase 3 - Ocorrências, imagens e localização
- ☐ Fase 4 - Confirmações e prioridade
- ☐ Fase 5 - Status, histórico e departamentos
- ☐ Fase 6 - Avaliações e contestação
- ☐ Fase 7 - Integração com IA
- ☐ Fase 8 - Dashboard e indicadores
- ☐ Fase 9 - Webhooks, n8n, Telegram e WhatsApp
- ☐ Fase 10 - Finalização e deploy

67. Checklist da Fase 0 - Inicialização

67.1 Projeto e dependências

- ☐ Criar package.json.
- ☐ Configurar versão recomendada do Node.js.
- ☐ Instalar dependências de produção e desenvolvimento.
- ☐ Criar scripts npm obrigatórios e npm run validate.
- ☐ Criar .gitignore e .env.example.
- ☐ Garantir ausência de segredos reais.

67.2 TypeScript

- ☐ Criar tsconfig.json.
- ☐ Ativar modo estrito.
- ☐ Configurar diretórios de origem e build.
- ☐ Garantir compilação.
- ☐ Garantir ausência de any e @ts-ignore indevidos.

67.3 Express

- ☐ Criar aplicação e servidor.
- ☐ Configurar JSON e limites de payload.
- ☐ Configurar request ID e respostas padronizadas.
- ☐ Criar tratamento centralizado de erros e rota inexistente.
- ☐ Implementar desligamento gracioso.

67.4 Variáveis e logs

- ☐ Validar ambiente com Zod.
- ☐ Validar NODE_ENV, PORT e DATABASE_URL.
- ☐ Impedir inicialização inválida.
- ☐ Configurar Pino, request ID e proteção de segredos.

67.5 PostgreSQL, PostGIS e Drizzle

- ☐ Criar Docker Compose.
- ☐ Configurar imagem PostGIS, volume e healthcheck.
- ☐ Expor porta 5432.
- ☐ Configurar Drizzle e cliente de banco.
- ☐ Confirmar conexão e extensão PostGIS.

67.6 Adminer

- ☐ Adicionar Adminer ao Docker Compose.
- ☐ Expor porta 8080.
- ☐ Documentar servidor postgres.

- ☐ Confirmar login.

67.7 Health

- ☐ Implementar GET /health com 200, timestamp e request ID.
- ☐ Implementar GET /health/database com 200 conectado.
- ☐ Retornar 503 quando indisponível.
- ☐ Padronizar erros.

67.8 Swagger

- ☐ Criar OpenAPI e Swagger UI em /docs.
- ☐ Disponibilizar /docs/openapi.json.
- ☐ Documentar as duas rotas de health.
- ☐ Adicionar exemplos e esquema JWT Bearer.
- ☐ Exibir botão Authorize.

67.9 Testes e qualidade

- ☐ Configurar Vitest, Supertest e cobertura.
- ☐ Testar health, banco, request ID, 404 e error handler.
- ☐ Testar banco indisponível.
- ☐ Configurar ESLint e Prettier.
- ☐ Executar lint, testes, cobertura, build e validate.
- ☐ Garantir ausência de funções vazias e TODO como implementação.

67.10 Coleção e README

- ☐ Escolher Bruno ou Postman.
- ☐ Criar ambiente local e requisições health.
- ☐ Documentar instalação, .env, Docker, migrations, Swagger, Adminer, testes, cobertura, validate, curl e solução de problemas.

67.11 Validação final

- ☐ API, Swagger, Adminer e PostgreSQL disponíveis nas portas definidas.
- ☐ PostGIS habilitado.
- ☐ Health da API e banco aprovados.
- ☐ Lint, testes, cobertura, build e validate aprovados.
- ☐ Fase aprovada pelo usuário.

68. Checklist da Fase 1 - Banco de dados

68.1 Enums e tabelas

- ☐ Criar enums de perfis, status, risco, origem, imagem, moderação, IA, notificação, webhook e outbox.

☐ Criar municipalities, neighborhoods, categories, departments, users, refresh_tokens, occurrences, occurrence_reports, occurrence_images, occurrence_confirmations, occurrence_status_history, repair_evaluations, ai_analyses, notifications, audit_logs, webhook_events, outbox_events e protocol_counters.

68.2 Relacionamentos, constraints e índices

- ☐ Criar chaves estrangeiras e restrições únicas.
- ☐ Criar checks de coordenadas, notas, gravidade, confiança e contadores.
- ☐ Impedir autorreferência de duplicidade.
- ☐ Configurar exclusão lógica e timestamps.
- ☐ Criar índices de e-mail, telefone, protocolo, município, bairro, categoria, status, data, GiST, confirmações e dashboard.

68.3 Migrations e seeds

- ☐ Criar migration inicial e extensão PostGIS.
- ☐ Executar migrations e conferir no Adminer.
- ☐ Criar municípios, bairros, categorias, departamentos, usuários preparados, ocorrências, históricos, confirmações, notificações e avaliações.
- ☐ Tornar seed idempotente e bloqueado em produção.
- ☐ Executar db:seed.

68.4 Validação

- ☐ Conferir tabelas, relacionamentos, índices, PostGIS e seeds no Adminer.
- ☐ Aprovar testes, Swagger, coleção, README e validate.
- ☐ Obter aprovação do usuário.

69. Checklist da Fase 2 - Autenticação e usuários

69.1 Usuários e credenciais

- ☐ Implementar cadastro, e-mail, telefone, duplicidade e Argon2id.
- ☐ Impedir escolha do perfil pelo cidadão.
- ☐ Implementar perfil e exclusão lógica.

69.2 Login, tokens e sessões

- ☐ Implementar login, access JWT e refresh opaco.
- ☐ Armazenar somente hash do refresh.
- ☐ Implementar cookie httpOnly, rotação e detecção de reutilização.
- ☐ Implementar logout, revogação, alteração de senha e /auth/me.
- ☐ Revogar sessões após troca de senha.

69.3 Middlewares e perfis

- ☐ Criar authenticate, authorizeRoles, authorizeMunicipality e autorização de proprietário.
- ☐ Tratar token ausente, inválido, expirado e usuário bloqueado.

- ☐ Criar seeds de todos os perfis, segundo município e usuário bloqueado.

69.4 Documentação e testes

- ☐ Documentar todas as rotas no Swagger e ativar Authorize.
- ☐ Testar 401, 403, 409 e 422.
- ☐ Testar perfis, outro município, revogação e reutilização de refresh.
- ☐ Conferir hashes e sessões no Adminer.
- ☐ Atualizar matriz, coleção e README.
- ☐ Aprovar validate e obter aprovação.

70. Checklist da Fase 3 - Ocorrências, imagens e localização

70.1 Ocorrências

- ☐ Implementar criação, protocolo, PENDING_REVIEW, report, histórico e auditoria.
- ☐ Implementar detalhes, listagens, filtros, paginação, exclusão lógica e privacidade.

70.2 Imagens

- ☐ Implementar abstração S3, multipart, JPEG/PNG/WebP, MIME real, limites, chave aleatória, metadados e limpeza de órfãos.
- ☐ Documentar upload no Swagger.

70.3 Localização

- ☐ Criar ponto PostGIS e índice.
- ☐ Implementar ST_DWithin, raio configurável, nearby e mapa.
- ☐ Validar coordenadas, município e privacidade pública.

70.4 Testes e validação

- ☐ Testar criação válida, sem imagem, coordenadas, MIME, tamanho, autenticação, 404, paginação, filtros, proximidade, privacidade e isolamento.
- ☐ Testar upload pelo Swagger e conferir ocorrência, report, imagem e localização no Adminer.
- ☐ Atualizar documentação, coleção e README.
- ☐ Aprovar validate e obter aprovação.

71. Checklist da Fase 4 - Confirmações e prioridade

- ☐ Implementar confirmação, prevenção de duplicidade, remoção e contador transacional.
- ☐ Impedir contador negativo e criar auditoria/notificação.
- ☐ Criar PriorityService com confirmações, gravidade, tempo, risco, pesos, limites e duas casas.
- ☐ Recalcular após inclusão, remoção e alterações relevantes.
- ☐ Criar rotina administrativa de recálculo.
- ☐ Testar primeira confirmação, duplicada, remoção, contador, prioridade, auditoria, notificação e permissões.
- ☐ Conferir dados no Adminer e testar Swagger.

- ☐ Atualizar documentação, coleção e README.
- ☐ Aprovar validate e obter aprovação.

72. Checklist da Fase 5 - Status, histórico e departamentos

- ☐ Implementar CRUD/ativação de departamentos e isolamento municipal.
- ☐ Criar máquina de estados com todas as transições.
- ☐ Impedir transições inválidas com 409.
- ☐ Validar perfil, município e campos por transição.
- ☐ Implementar atribuição, previsão, agendamento, início, resolução, fechamento, contestação e reabertura.
- ☐ Criar histórico, auditoria e notificações dentro de transação.
- ☐ Testar todos os perfis, outro município, rollback e registros gerados.
- ☐ Conferir Adminer, Swagger, matriz, coleção e README.
- ☐ Aprovar validate e obter aprovação.

73. Checklist da Fase 6 - Avaliações e contestação

- ☐ Implementar criação, notas, relação do usuário, status permitido e unicidade.
- ☐ Permitir edição no prazo e resumo.
- ☐ Calcular proporção negativa e mínimo de avaliações.
- ☐ Alterar para CONTESTED ou revisão administrativa.
- ☐ Criar histórico, auditoria e notificações.
- ☐ Testar avaliação válida, duplicada, usuário não relacionado, status inválido, nota, prazo, contestação e reabertura.
- ☐ Conferir Adminer, Swagger, coleção e README.
- ☐ Aprovar validate e obter aprovação.

74. Checklist da Fase 7 - Integração com IA

- ☐ Criar cliente HTTP com URL, segredo, timeout e tentativas.
- ☐ Proteger segredo e tratar indisponibilidade/resposta inválida.
- ☐ Enviar report, imagem, descrição, localização, próximas e categorias.
- ☐ Validar categoria, subcategoria, gravidade, risco, confiança, revisão e duplicidades.
- ☐ Salvar resposta bruta e validada.
- ☐ Aplicar fallback e manter ocorrência registrada.
- ☐ Impedir acesso direto ao banco, alteração de status e fusão automática.
- ☐ Testar válido, inválido, timeout, indisponibilidade, confiança baixa, duplicidade, retry e fallback.
- ☐ Conferir Adminer e documentação.
- ☐ Aprovar validate e obter aprovação.

75. Checklist da Fase 8 - Dashboard e indicadores

- ☐ Implementar resumo, categoria, bairro, status, prioridade, resolução, mapa de calor e exportação do MVP.
- ☐ Aplicar filtros de período, categoria, bairro, status, município e perfil.
- ☐ Proteger dados pessoais, evitar N+1 e criar índices.
- ☐ Testar dados vazios, filtros, agregações, municípios, perfis, desempenho e exportação.
- ☐ Comparar resultados com Adminer.
- ☐ Atualizar Swagger, coleção e README.
- ☐ Aprovar validate e obter aprovação.

76. Checklist da Fase 9 - Webhooks e n8n

- ☐ Validar segredo, assinatura, rate limit, idempotência e hash do payload.
- ☐ Impedir duplicidade, auditar e remover segredos dos logs.
- ☐ Implementar webhooks Telegram, WhatsApp, status e n8n.
- ☐ Implementar webhook_events, outbox_events, estados, tentativas e falhas.
- ☐ Testar assinatura, duplicidade, payload, rate limit, processamento, falha, retry e logs.
- ☐ Conferir eventos e outbox no Adminer.
- ☐ Atualizar Swagger, coleção e README.
- ☐ Aprovar validate e obter aprovação.

77. Checklist da Fase 10 - Finalização

77.1 Testes finais

- ☐ Executar todos os testes, cobertura e fluxos completos.
- ☐ Revisar módulos abaixo da meta.
- ☐ Testar permissões, isolamento, falhas externas, migrations e seeds em banco limpo.

77.2 Documentação e segurança

- ☐ Revisar todas as rotas, schemas, exemplos, códigos, autenticação, multipart e erros no Swagger.
- ☐ Revisar coleção, ambientes e remover tokens reais.
- ☐ Revisar README completo e solução de problemas.
- ☐ Revisar segredos, CORS, Helmet, rate limit, logs, upload, SQL, autorização, isolamento, exclusão lógica e LGPD.
- ☐ Criar diagrama ER, contratos da IA/painel/n8n e relatório de segurança.
- ☐ Documentar limitações.

77.3 Validação final

- ☐ Executar lint, unit, integration, coverage, build e validate.
- ☐ Subir Docker, migrations e seeds.
- ☐ Testar Swagger, Adminer, coleção e curl.

- ☐ Confirmar ausência de segredos, funções vazias e TODO como implementação.
- ☐ Obter aprovação final do usuário.

78. Modelo obrigatório de atualização

Ao final de toda resposta de desenvolvimento:

Fase atual: Fase X - Nome

[x] Item concluído

Evidência: validação objetiva.

[~] Item em desenvolvimento

Evidência pendente: o que falta validar.

[!] Item bloqueado

Motivo: explicação.

Ação necessária: comando ou informação.

[] Item não iniciado

Resumo:

Itens concluídos: X

Itens em desenvolvimento: X

Itens bloqueados: X

Itens não iniciados: X

Progresso estimado da fase: X%

Itens fora do escopo não entram no cálculo.

79. Regra de passagem entre fases

A IA não poderá iniciar a fase seguinte enquanto:

houver itens obrigatórios não iniciados ou bloqueados;

o usuário não enviar resultados solicitados;

npm run validate não estiver aprovado;

migrations e testes manuais não estiverem validados;

a fase não tiver aprovação explícita do usuário.

Ao concluir:

Fase X tecnicamente concluída e aguardando aprovação do usuário.

Somente após aprovação:

[x] Concluída e aprovada

80. Regra contra conclusões falsas

É proibido marcar concluído apenas porque o código foi sugerido, um arquivo foi exibido, o comando deveria funcionar, o teste foi escrito mas não executado, a migration foi criada mas não executada ou a rota foi documentada mas não testada.

Sem comprovação, utilizar:

[~] Implementado, aguardando validação

Apêndice A - Portas e acessos locais

API: <http://localhost:3333>

Swagger: <http://localhost:3333/docs>

OpenAPI JSON: <http://localhost:3333/docs/openapi.json>

Adminer: <http://localhost:8080>

PostgreSQL: localhost:5432

Apêndice B - Comandos principais

`npm install`

`cp .env.example .env`

`docker compose up -d`

`npm run db:migrate`

`npm run db:seed`

`npm run lint`

`npm run test:unit`

`npm run test:integration`

`npm run test:coverage`

`npm run build`

`npm run validate`

`npm run dev`